Architecture & Principles

The architecture, the OOP patterns it leans on, and the conventions that run through the code.

Finance Portal is built on SOLID and a layered (Clean) architecture, with a focused set of patterns that keep the modules decoupled and the system resilient. This is a short tour of how it fits together and the reasoning behind the main decisions.

Layered architecture

Requests flow through clear layers; dependencies always point inward:

```
Controller  →  Service  →  Repository (JPA)  →  PostgreSQL
    |             |
ApiResponse     Ports:  AssetPricingPort · PortfolioSnapshotPort ·
                        EventPublisherPort · UserStatusPort
```

- **Multi-module by domain.** `finance-market`, `finance-portfolio`, `finance-user`, `finance-news` each own a slice; `finance-common` holds what both backends share (response envelopes, security, events, rate limiting). The **notification microservice** (`:8082`) is a separate Spring Boot app with **zero** dependency on monolith-only code — the two communicate only through Kafka events and the shared database.
- **Ports & adapters.** Cross-module and cross-backend calls go through ports (`AssetPricingPort` , `PortfolioSnapshotPort` , `EventPublisherPort` , `UserStatusPort`), so a module depends on an interface, not on another module's internals.
- **CQRS-lite.** Reads and writes are split into separate services (`TrackedAssetQueryService` / `TrackedAssetCommandService`) instead of one god-service.

Shared library (`finance-common`)

Both runtimes — the core backend and the notification microservice — depend on one shared Maven module, `finance-common` . It is the single place cross-cutting contracts live, so the two apps cannot quietly drift apart:

- **API surface** — `ApiResponse<T>` , `PagedResponse<T>` , `ErrorResponse` and the `GlobalExceptionHandler` , so every endpoint on either backend answers in the same envelope.
- **Event vocabulary** — the `DomainEvent` records, `KafkaTopics` and `EventPublisherPort` : the only language the two services use to talk to each other over Kafka.
- **Security & limits** — the OAuth2 resource-server config, the Bucket4j `RateLimitFilter` and its tier strategy, and the `UserStatusPort` / `DbUserStatusAdapter` , so both backends authenticate and throttle by exactly the same rules.

- **Shared domain & infra** — `MarketType`, the `TrackedAsset` model, `AssetSnapshotCache`, `AppProperties`, and the Redis / WebClient base config.

It is a plain library, not a service: each backend compiles it into its own image. The Docker build installs `finance-common` from source as its first layer (`backend/Dockerfile`), so a fresh clone builds the whole stack with no extra step. A dedicated CI workflow (`publish-finance-common.yml`) also pushes it to **GitHub Packages (Maven)** as `com.finance:finance-common` whenever `finance-common/` changes on `main`, so external repos in the same org can pull it as a versioned Maven dependency. The publish is a sharing convenience, never a prerequisite for building or running locally.

SOLID in practice

- **S** — single responsibility: the Keycloak admin client is split into client / token manager / exception translator; query and command services are separate.
- **O** — open/closed: adding a market type means a new `MarketAssetProvider` strategy, not editing a switch.
- **L** — Liskov: `BaseAsset` / `BaseCandle` subtypes are interchangeable wherever the base is used.
- **I** — interface segregation: narrow ports (`AssetPricingPort`, `UserStatusPort`) instead of one fat service interface.
- **D** — dependency inversion: high-level modules depend on ports; adapters wire the concretions.

OOP & design patterns

- **Enum carries behaviour.** `MarketType`, `AssetType`, `FundType`, `Tier`, … define abstract methods on their constants. The codebase **never** `switch`**es** on these enums — it adds a method.
- **Strategy +** `EnumMap` **dispatch** — `MarketAssetProvider`, `SnapshotBatchRefresher`, `CandleBatchRefresher`. Event publishing reads `event.topic()` / `event.partitionKey()` rather than branching on type.
- **Sealed types over maps** — `MarketAssetMetadata` / `NotificationPayload` are sealed interfaces with record implementations, so `Map<String, Object>` stays off the response surface.
- **Observer via events** — schedulers publish domain events to Kafka; the notification service reacts. Producers don't know their consumers.

Resilience & external integrations

External data — CoinGecko, Binance, Yahoo Finance, TEFAS, TCMB/EVDS, doviz.com (FX & gold, scraped with **jsoup**), İş Yatırım (VIOP / derivatives) and the Keycloak admin API — is reached over Spring `WebClient` on scheduled jobs, hardened with **Resilience4j**:

- **Retry** with backoff on transient failures, so a single dropped response doesn't fail a refresh.

- **Circuit breaker** around each upstream client (e.g. `CoinGeckoClient`, the bond/fund fetchers, `KeycloakAdminClient`) — when a provider is down the breaker opens and the app fails fast and degrades gracefully instead of hammering a dead dependency or cascading the failure.
- A failed provider surfaces as `503 EXTERNAL_API_ERROR` (see *Error Handling*), never as a 500.
- Historical FX is applied **per candle at its own date's rate**, never today's spot for a past date — so multi-currency P&L and comparisons stay correct over time.

Caching strategy

Two layers, each for a different job:

- **Redis — cross-request / cross-instance state, shared by *both* backends.** It holds far more than rate-limit buckets: market snapshot caches, the **gainers/losers (top-movers)** behind the overview page (`TopMoversRedisService`), per-asset **detail-page** snapshots, the inflation-**beater** results, and the rendered **overview widgets** — so the dashboard and hot reads never recompute or re-hit the database. The **Bucket4j rate-limit buckets** also live in Redis (keyed `rate-limit:{userSub}:{tier}`), which is exactly why the core backend and the notification service enforce one shared set of limits.
- **Caffeine — in-process hot reads.** `TrackedAssetCodeCache`, the historical-FX-rate adapter, the beater cache manager and the portfolio backfill tracker use Caffeine for sub-millisecond local lookups.
- `ConcurrentHashMap` is confined to `TaskTrackingService`; every other cache-like structure goes through Caffeine or Redis.

Rate limiting

- **Bucket4j**, Redis-backed, drives per-user token buckets keyed `rate-limit:{userSub}:{tier}` (`RateLimitFilter` in `finance-common`), with tiers for normal, auth-sensitive and admin-trigger traffic. Because the key naming is shared, both backends honour the same buckets.
- The **Nginx** edge adds coarse zones on top (auth ~5 r/s, api ~50 r/s, static ~200 r/s) plus a per-IP connection limit.

Observability

- An **OpenTelemetry Java agent runs in *both* backends** (downloaded in each Dockerfile), exporting traces over OTLP/gRPC to the collector and on to OpenSearch.
- Application logging uses **Log4j2 with a JSON layout** (replacing Logback); logs are shipped through Kafka into OpenSearch, so logs, traces and metrics are queryable together in OpenSearch Dashboards.

Security

Stateless JWT auth via Spring **OAuth2 Resource Server**, validated against the Keycloak realm's JWKS — no server session. CSRF is disabled (token-based, not cookie-based), CORS is configurable (`AppProperties.Cors`), and `/api/v1/admin/**` is gated by the `ADMIN` role; Swagger, health and auth endpoints stay public.

Frontend

- **Server state in TanStack Query, client state in Zustand** — server data is never duplicated into Zustand; queries own caching, refetch and invalidation.
- React 19 + **Rolldown-Vite** + **Tailwind CSS 4**, plain JavaScript (no TypeScript); motion via Framer Motion, shared UI primitives over inline one-offs.
- Charts use **TradingView Lightweight Charts** (price/candles, with a custom drawing layer) plus **ECharts** (allocation / analytics). The UI is fully **bilingual (TR/EN)** via i18next, and the overview is a customizable drag-and-drop widget dashboard (react-grid-layout + dnd-kit).

Conventions

A few conventions run through the code:

- Dependencies are injected through constructors (Lombok `@RequiredArgsConstructor`) rather than field `@Autowired` .
- Domain code stays free of reflection.
- Controllers return an `ApiResponse<T>` and don't expose entities directly.
- Public APIs carry Javadoc (JSDoc on the frontend); comments explain *why* something is done, not *what* the code already says.
- Money is `BigDecimal` (scale 4 for amounts, 8 for quantities), and the portfolio stores values in TRY, converted at the trade date's historical rate.

Data & migrations

- PostgreSQL with **Flyway**; Hibernate runs `ddl-auto: validate` — schema changes are migrations, never auto-generated. Financial columns are `precision = 19` .
- Flyway owns the schema and seeded config rows (fixed IDs). Runtime/market data lives in the app and, for the clone-and-run demo, in the `seed/` snapshot — which touches data tables only.